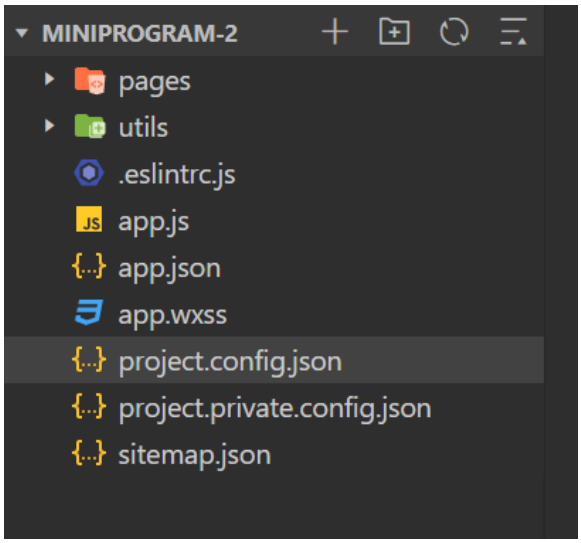
小程序知识总结

1、项目结构和文件

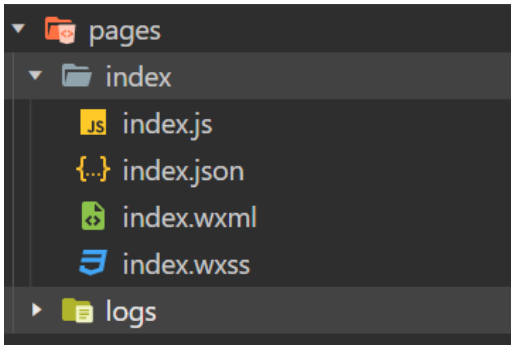
1、项目结构

在创建小程序之后，出现的项目结构是这样的：



pages文件夹下是每一个页面，utils是一些工具性质的模块，app.js是程序的主入口文件，app.json是全局配置文件，app.wxss是全局样式文件。project.config.json是项目的配置文件，sitemap.json是表示小程序是否被微信索引找到。

每一个pages中的文件夹又是这样的：



js表示一些操作和数据，json表示本页面的配置信息，wxml表示显示的页面，wxss表示样式文件。

2、全局JSON，JS，WXSS的配置

1. 全局JSON：

```
"pages":[
  "pages/index/index",
  "pages/logs/logs"
],
"window":{
  "backgroundTextStyle":"light",
  "navigationBarBackgroundColor": "#fff",
  "navigationBarTitleText": "Weixin",
  "navigationBarTextStyle":"black"
},
"style": "v2",
"sitemapLocation": "sitemap.json"
```

pages表示所有的页面，windows表示小程序界面的配置，在之后的，还有tabBar和usingComponents的配置：

```
"tabBar": {
  "color": "#8a8a8a",
  "selectedColor": "#01F8E4",
  "backgroundColor":"#efefef",
  "list": [{…
  },{…
  },{…
  }]
},
"style": "v2",
"sitemapLocation": "sitemap.json",
"usingComponents": {
  "MyHeader":"/components/MyHeader/MyHeader",
  "MyFooter":"/components/MyFooter/MyFooter"
}
```

tabBar是指在页面中的

存放在List中，

```
"list": [{
  "text": "首页",
  "pagePath": "pages/index/index",
  "iconPath": "static/images/icons/home.png",
  "selectedIconPath": "static/images/icons/home-fill.png"
```

usingComponents是指使用到的组件。（注意区分pages和Components)的区别。

2. app.js

```
// app.js
App({
  onLaunch() {
    // 展示本地存储能力
    const logs = wx.getStorageSync('logs') || [
    logs.unshift(Date.now())
    wx.setStorageSync('logs', logs)

    // 登录
    wx.login({
      success: res => {
        // 发送 res.code 到后台换取 openId,
        sessionKey, unionId
      }
    })
  },
  globalData: {
    userInfo: null
  }
})
```

项目通过调用App()函数启动本程序。

3、app.wxss

通过这样的方式进行全局的样式配置

3、pages文件

pages文件由四部分组成，和全局的很像，这里主要对比区别：

1. js文件

app.js通过App()，pages通过Page()来调用。从结构来讲，其中都含有data:{}(数据部分)和处理事件的方法。

```
Page({
  data: {
    motto: 'Hello World',
    userInfo: {},
    hasUserInfo: false,
    canIUse: wx.canIUse('button.open-type.getUserInfo'),
    canIUseGetUserProfile: false,
    canIUseOpenData: wx.canIUse('open-data.type.userAvatarUrl') && wx.canIUse('open-data.type.userNickName') // 如需尝试获取用户信息可改为false
  },
  // 事件处理函数
  bindViewTap() {
    wx.navigateTo({
      url: '../logs/logs'
    })
  },
  onLoad() {
    if (wx.getUserProfile) {
      this.setData({
```

2. json文件

pages > api2 > {..} api2.json > ...

```
{
  "usingComponents": {},
  "enablePullDownRefresh":true,
  "backgroundColor":"#00804D",
  "backgroundTextStyle":"light",
  "onReachBottomDistance":200
}
```

这些参数可以通过查找文档来找到：

小程序配置 / 页面配置 (qq.com)

- 小程序配置
  - 全局配置
  - 页面配置
  - sitemap 配置
- 场景值
- 框架接口
- WXML 语法参考
- WXS 语法参考

小程序配置 / 页面配置

页面配置

app.json 中的部分配置，也支持对单个页面进行配置，可以在页面对应的 `.json` 文件来对本页面的表现进行配置。

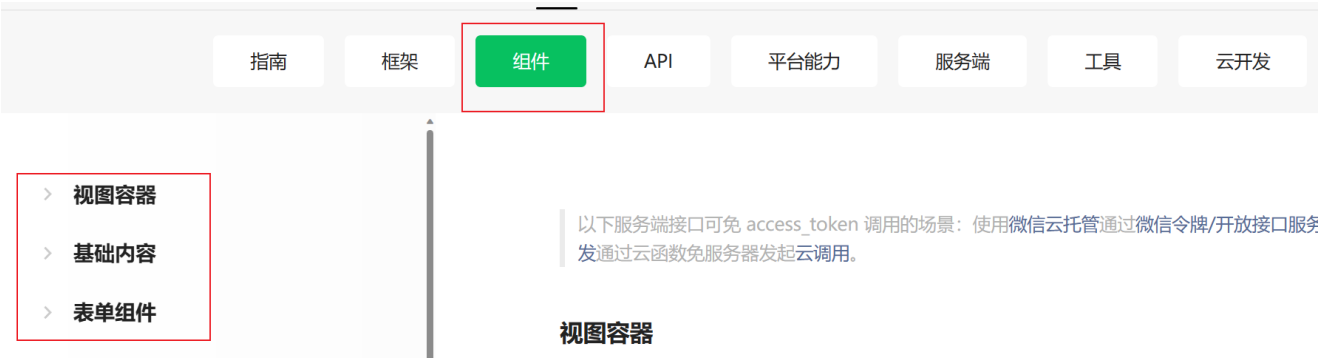
页面中配置项在当前页面会覆盖 `app.json` 中相同的配置项（样式相关的配置项属于 `app.json` 中的 `window` 属性，但这里不需要额外指定 `window` 字段），具体的取值和含义可参考全局配置文档中说明。

文件内容为一个 JSON 对象，有以下属性：

配置项

属性	类型	默认值	描述	最低版
navigationBarBackgroundColor	HexColor	#000000	导航栏背景颜色，如 `#000000`	
navigationBarTextStyle	string	white	导航栏标题、状态栏颜色，仅支持 `black` / `white`	

小程序组件



小程序配置 / 页面配置 (qq.com)

这部分相当于HTML，可以直接学习。

小程序接口

表单组件 / button (qq.com)

主要分为：

- 监听事件api，on+
- 同步api，sync结尾
- 异步api，需要success、fail来调用结果的。、

2、模板语法

1.数据绑定

这部分主要的概念就是：如何使用data中的数据。

- 绑定内容:使用 `{{}}` 来获得值。

```
我的名字是：{{name}}



page({
    data: {
    name:"张三"
    }
})
```

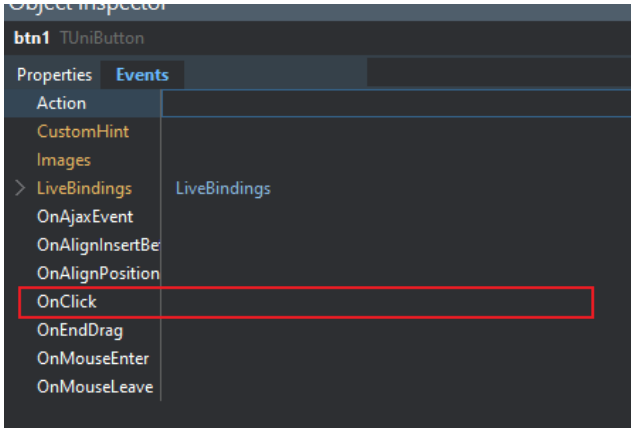
- 绑定属性

```
page({
    data:{
        imagesrc:"http://ssss"
    }
})

<image src="{{imagesrc}}"><image/>
```

2.事件绑定

常见的事件绑定，在delphi中是通过方法来完成的：



小程序的事件是指：渲染层到逻辑层的通信方式。

我们只需要在需要触发的div上绑定方法，再到Js中写出对应的方法即可。常见的事件有：

- bindtap：手指触摸后马上离开，类似HTML中的click事件。
- bindinput：输入框输入事件。
- change：bindchange：状态改变时触发。

```
    <button type="primary" bindtap="outerhander">
        按钮
    </button>
```

对应的JS文件的事情处理函数是这样的：

```
page({
    outerhander(e){ //按钮tap事件的处理函数
        console.log(e) // 事件参数对象e
    }
})
```

在点击时候会出现冒泡的问题，可以查看该链接的讲解和处理：

https://www.jianshu.com/p/9a4da00987c1

并且在示例代码中的pages-index中有示例代码。

3.事件传值和Js中的data重新设值

这两者可以称之为小程序开发的异常 '反人类'的地方。

对于vue，事件的传值只需要在事件的（）中填上就可以了，比如add(1,2)。并且随后的js中的Data直接通过赋值语句就可以实现data中值的变化。

```
<button type="primary" bindtap="btnHandler(123)">
    传值事件
</button>
```

在小程序中，相当于调用一个名字 `btnHandler(123)` 的方法。

如何想要传值可以这样写：

```
<button bindtap="btnHandler" data-info="{{123}}">
事件传值
</button>
```

`info` 会被解析为参数的名字，数值123会被解析为参数的值。

js通过 `event.target.dataset.参数名` 来获得具体的值。

```
btnHandler(event){
    // dataset 是一个对象，包含了所有通过data-* 传递来的参数项
    console.log(event.target.dataset)
    console.log(event.target.dataset.info)
}
```

同时数据的重新设值是这样的：

```
<input placeholder="{{mes}}" bindinput="inher"/>
```

```
 inher(event){
    this.setData({
      mes:event.detail.value
    })
    console.log(event.detail.value);
  },
```

4.双向绑定

双向绑定的含义就是，渲染层改变，逻辑层中的data也改变，也就是：wxml改变，js中的也改变。其实上边的例子是一个手动的双向绑定，也可以根据其设置自动的双向绑定：

```
<input model:value="{{value}}" bindinput="inher1" />
```

```
 inher1(){
    console.log(this.data.value);
  },
```

通过 `model:value=` 来绑定data中的数值，注意，绑定只能是单一的字段绑定，并且不能data路径。

比如

```
<input model:value="值为 {{value}}" />
<input model:value="{{ a + b }}" />


<input model:value="{{ a.b }}" />
```

5.条件渲染

就是通过一个属性来控制该组件是否渲染出来，有两种：1、渲染出来隐藏。2、不渲染出来。

通过

```
<view wx:if="{{condition}}"> True </view>这样来判断该组件是否渲染出来。
```

支持多个条件判断(不建议使用)

```
<view wx:if="{{length > 5}}"> 1 </view>
<view wx:elif="{{length > 2}}"> 2 </view>
<view wx:else> 3 </view>
```

而 `wx:hidden` 的含义就是：组件始终会被渲染，只是简单控制显示和隐藏。

6.列表渲染

如果是一个数组，数组中的每个对象都要渲染出来，使用 `wx:for`

```
<view wx:for="{{array}}" wx:key="id">
    {{index}}:{{item.message}}
</view>
```

```
Page({
    data:{
        array:[{id:'1',message:'foo'},{id:'2',message:'bar'}]
    }
})
```

这其中有四个东西在里面：

- array：被渲染的数组
- index：渲染数组的唯一标识
- item：当前渲染的数组中的一个对象
- key：唯一标识（如果没有会造成性能问题，功能不会出问题）

3、网络请求和Promise封装

这部分建议学习一下Promise的相关文档。

看这篇文章旧可以了。

https://www.bilibili.com/read/cv18799030/

4、路由和pages

1、路由导航

一般来说不同页面之间需要跳转，小程序提供了两种方式：

- 声明式导航
- 编程式导航

两者的区别是这样的：

1. 声明式是通过组件 `<navigator>` 。
2. 编程式导航通过调用小程序的导航api，也就是在js中的方式导航。

1)：声明式导航

- 导航到TabBar页面（就是配置了tabBar的页面）

  ```
  <navigator url="/pages/……" open-type="switchTab">导航到tabbar页面 </navigator>
  ```

  - url：需要跳转的页面，必须以 `/` 开头。
  - open-type：标识跳转的方式，必须为switchTab。
- 导航到非TabBar页面

  ```
  <navigator url="/pages/……" open-type="navigate">导航到非tabbar页面 </navigator>
  ```

  - 注意，此处open-type可以省略。
- 后退导航

  ```
  <navigator u open-type="navigateBack" delta='1'>返回上一页</navigator>
  ```

  - delta必须是数字，默认第一层不需要填写

2):编程导航

- 导航到tabBar页面。调用wx.switchTab（Object object）方法，object参数对象的属性列表如下：

属性	类型	是否必选	说明
url	string	是	需要跳转的 tabBar 页面的路径，路径后不能带参数
success	function	否	接口调用成功的回调函数
fail	function	否	接口调用失败的回调函数
complete	function	否	接口调用结束的回调函数（调用成功、失败都会执行）

```
<button bindtap="gotoMessage">跳转到消息页面</button>
```

```
gotoMessage(){
    wx.switchTab({
        url:'/pages/message/message'
    })
}
```

- 导航到非tabBar页面，调用wx.navigateTo(Object object)跳转到非tabBar页面。参数如下：

属性	类型	是否必选	说明
url	string	是	需要跳转的 tabBar 页面的路径，路径后能带参数
success	function	否	接口调用成功的回调函数
fail	function	否	接口调用失败的回调函数
complete	function	否	接口调用结束的回调函数（调用成功、失败都会执行）

```
<button bindtap="gotoInfo">跳转到消息页面</button>
```

```
gotoInfo(){
    wx.navigateTo({
        url:'/pages/info/info'
    })
}
```

- 后退导航，调用wx.navigateBack(Object object)方法，可以返回上一页面或多级页面，参数如下

属性	类型	是否必选	说明
delta	number	否	默认值为1，表示返回的页面数，如果delta大于现有页面数，那就返回首页。
success	function	否	接口调用成功的回调函数
fail	function	否	接口调用失败的回调函数
complete	function	否	接口调用结束的回调函数（调用成功、失败都会执行）

```
<button bindtap="gotoBack">后退</button>
```

```
gotoBack(){
    wx.navigateBack()
}
```

3):导航传参

- 声明式导航传参，在路径的后面可以携带参数：

```
<navigator url="/pages/info/info?name=wrf&age=20">跳转到info页面 </navigator>
```

- 编程式导航传参，使用navigateTo方法，在url中填写参数。

```
gotoMessageNoTab(){
    wx.navigateTo({
      url: '/pages/info/info?name=三大爷&age=50',
    })
  }
```

- 接收参数

在 `onload` 方法中接收参数

```
  onLoad(options) {
    console.log(options)
  }
```

`注意`：下面将会讲解onload的方法，这涉及到小程序的生命周期。

2、生命周期

生命周期是指：一个东西，从产生到消亡的周期。

小程序的生命周期分为两个部分：小程序应用的生命周期和页面的生命周期。

1. 小程序应用的生命周期

在app.js中，代码如下：

```
App({
  onLaunch: function () {
    console.log("onLaunch初始化了项目");
    this.globalData.name='咸虾米'
  },

  onShow: function (options) {
    console.log("onShow触发了");
  },

  onHide: function () {
    console.log("onHide触发了");
  },

  onError: function (msg) {

  }
})
```

- onLaunch：小程序初始化完成时，执行此函数，全局只触发一次，可以做一些初始化工作
- onshow：小程序启动，或者从后台进行前台显示时触发
- onHide:小程序从前台进入后台时触发。

2. 页面的生命周期

在页面js中

```
Page({
    onLoad //监听页面加载，一个页面只调用一次。
    onshow //监听页面显示
    onready //  监听页面初次渲染完成，一个页面只调用1次
    onHide //监听页面隐藏
    onUnload //监听页面卸载，一个页面只调用1次
})
```

3、页面事件

- 下拉刷新 `onPullDownRefresh()` ,在json文件中，将 `enablePullDownRefresh` 设置为true。有全局配置和页面配置的区分。
- 关闭下拉效果： `wx.stopPullDownRefresh()` 。
- 上拉触底： `onReachBottom()` ,通过json文件中的 `onReachBottomDistance` .

5、自定义组件

基本概念

自定义组件是什么：组件可以某种意义上等同于uniGUI的frame，这是一个单独封装的组件内容。可以实现样式的隔离和组件的重复利用。

在项目根目录下创建一个components文件夹，然后再其下创建文件夹，表示组件的名字。再如下图创建Component文件。

当我们创建了自定义组件的时候，就需要在 `pages/components` 中使用这块组件。在页面的JSON文件中，使用 `usingComponents` 配置需要引入的文件。

引入分为 `全局引入` 和 `按需引入` 。

- 全局引入：就是直接在 `app.json` 中引入，所引入的组件可以供所有的页面使用。
- 按需引入：在需要用的pages中的引入，也只能在本pages中使用。

`app.json` 代码:

```
{
  "pages":[
    "pages/index/index",
    "pages/search/search",
    "pages/classify/classify",
    "pages/news/news",
    "pages/newsDetail/newsDetail",
    "pages/proDetail/proDetail"
  ],
  "window":{
  },
  "usingComponents": {
      // 这是自定义组件
    "xzs-header":"components/xzs-header/xzs-header",
    "xzs-news-item":"components/xzs-news-item/xzs-news-item",
    "xzs-footer":"components/xzs-footer/xzs-footer",
    "xzs-product-item":"components/xzs-product-item/xzs-product-item",

      // 这是npm安装的组件
    "van-button": "@vant/weapp/button/index",
    "van-icon": "@vant/weapp/icon/index",
    "van-loading": "@vant/weapp/loading/index",
    "van-skeleton": "@vant/weapp/skeleton/index",
    "van-tab": "@vant/weapp/tab/index",
    "van-tabs": "@vant/weapp/tabs/index"
  }
}
```

页面的JS文件内容

1. data：数据内容。
2. methods：事件方法。
3. properties：对外属性，用来接收外界传递到组件中的数据。

`注意`：在自定义组件中，方法建议使用 `_` 开头

```
Component({
  /**
   * 组件的属性列表
   */
  properties: {

  },

  /**
   * 组件的初始数据
   */
  data: {
    year:new Date().getFullYear()
  },

  /**
   * 组件的方法列表
   */
  methods: {

  }
})
```

组件和页面的区别

主要JSON文件和JS文件含有些微的差别：

- 组件的JSON需要声明 `"component": true,`
- 组件的Js文件调用的是Component方法。
- 组件处理事件的函数需要定义到 `methods` 中。

`注意`：一般情况下，自定义组件是 `样式隔离` 的，防止组件内外样式相互干扰，但是有时候我们希望在外界能够控制组件内部样式，可以通过 `stylesolation` 来控制样式。

- 在组件JS文件中新增如下配置

```
Component({
    options:{
        stylesolation:"islated"
    }
})
```

- 在JSON文件中新增如下配置

```
{
    stylesolation:"islated"
}
```

`stylesolation` 的参数列表为：

- isolated：启用样式隔离，
- apply-shared：表示页面wxss样式会影响到自定义组件，单自定义组件中的样式不会印象页面。
- shared：双向影响。

数据监听

用于监听和响应任何属性和数据字段的变化

- 监听字段的变化

```
  observers:{
    '字段A，字段B':function(A的新值，B的新值){
      // do something
    }
  },
```

- 监听对象中的单个或多个属性变化

```
 observers:{
    '对象.属性A，对象.属性B':function(A的新值，B的新值){
      // do something
    }
  },
```

- 监听对象中所有的属性变化

```
 observers:{
    '对象.**':function(obj){
      // do something
    }
  },
```

纯数据字段

纯数据字段是指：有些情况下，某些 `data` 中的字段（包括 `setData` 设置的字段）既不会展示在界面上，也不会传递给其他组件，仅仅在当前组件内部使用。

设置方法：在 `Component` 构造器的 `options` 定义段中指定 `pureDataPattern` 为一个正则表达式，字段名符合这个正则表达式的字段将成为纯数据字段。

```
Component({
  options: {
    pureDataPattern: /^_/ // 指定所有 _ 开头的数据字段为纯数据字段
  },
  data: {
    a: true, // 普通数据字段
    _b: true, // 纯数据字段
  },
  methods: {
    myMethod() {
      this.data._b // 纯数据字段可以在 this.data 中获取
      this.setData({
        c: true, // 普通数据字段
        _d: true, // 纯数据字段
```

```
      })
    }
  }
})
```

组件的生命周期

组件的生命周期，指的是组件自身的一些函数，这些函数在特殊的时间点或遇到一些特殊的框架事件时被自动触发。

其中，最重要的生命周期是 `created` `attached` `detached` ，包含一个组件实例生命流程的最主要时间点。

- 组件实例刚刚被创建好时， `created` 生命周期被触发。此时，组件数据 `this.data` 就是在 `Component` 构造器中定义的数据 `data` 。 **此时还不能调用 `setData` 。** 通常情况下，这个生命周期只应该用于给组件 `this` 添加一些自定义属性字段。
- 在组件完全初始化完毕、进入页面节点树后， `attached` 生命周期被触发。此时， `this.data` 已被初始化为组件的当前值。这个生命周期很有用，绝大多数初始化工作可以在这个时机进行。
- 在组件离开页面节点树后， `detached` 生命周期被触发。退出一个页面时，如果组件还在页面节点树中，则 `detached` 会被触发。

生命周期方法可以直接定义在 `Component` 构造器的第一级参数中。

代码示例

```
Component({
  lifetimes: {
    attached: function() {
      // 在组件实例进入页面节点树时执行
    },
    detached: function() {
      // 在组件实例被从页面节点树移除时执行
    },
  },
  // 以下是旧式的定义方式，可以保持对 <2.2.3 版本基础库的兼容
  attached: function() {
    // 在组件实例进入页面节点树时执行
  },
  detached: function() {
    // 在组件实例被从页面节点树移除时执行
  },
  // ...
})
```

可用的全部生命周期如下表所示。

生命周期	参数	描述	最低版本
created	无	在组件实例刚刚被创建时执行	1.6.3
attached	无	在组件实例进入页面节点树时执行	1.6.3
ready	无	在组件在视图层布局完成后执行	1.6.3
moved	无	在组件实例被移动到节点树另一个位置时执行	1.6.3
detached	无	在组件实例被从页面节点树移除时执行	1.6.3
error	`Object Error`	每当组件方法抛出错误时执行	2.4.1

页面的组件的生命周期

还有一些特殊的生命周期，它们并非与组件有很强的关联，但有时组件需要获知，以便组件内部处理。这样的生命周期称为“组件所在页面的生命周期”，在 `pageLifetimes` 定义段中定义。其中可用的生命周期包括：

生命周期	参数	描述	最低版本
show	无	组件所在的页面被展示时执行	2.2.3
hide	无	组件所在的页面被隐藏时执行	2.2.3
resize	`Object Size`	组件所在的页面尺寸变化时执行	2.4.0
routeDone	无	组件所在页面路由动画完成时执行	2.31.2

`注意`：自定义 tabBar 的 pageLifetime 不会触发。

```
Component({
  pageLifetimes: {
    show: function() {
      // 页面被展示
    },
    hide: function() {
      // 页面被隐藏
    },
    resize: function(size) {
      // 页面尺寸变化
    }
  }
})
```

组件父子通信

- 属性绑定（父传子）

只能传递普通类型的数据，不能传递方法给子组件

```
data:{
    count:0
}
// 父组件的xml结构
<my-test count="{{count}}"></my-test>
```

子组件的声明如下：

```
properties:{
    count:Number
}
<text>子组件中，count的值为：{{count}}</text>
```

- 事件绑定（子传父）可以传递任何类型的数据。其实是通过回调函数的方式来实现的。
    1. 在 `父组件` 中自定义方法。此方法将作为 `回调函数的方式` 传递给 `子组件` 。

    ```
    SyncCount(){
        console.log('1111')
    }
    ```

    2. 在父组件的wxml中，将步骤1的函数引用传递给子组件。

    `注意` `‘bind：’` 之后的表示子组件中 `triggerEvent的第一个参数` ,第二个表示 `回调函数` 。

    ```
    <mytest count="{{count}}" bind:ChildCount="SyncCount"></mytest>
    ```

3. 在子组件中，通过调用 `this.triggerEvent`("自定义事件名"，{参数对象})，将数据发送给父组件

```
// 子组件中的wxml结构
<text> 子组件中，count的值为{{count}}</text>
<button type="primary" bindtap="addCount">+1</button>

// 子组件中的js代码
methods：{
    addCount(){
        this.setData({
            count:this.properties.count+1
        })
        this.triggerEvent('ChildCount',{value:this.properties.count})
    }
}
```

4. 父组件的回调函数获得传递过来的值。

```
SyncCount(e){
    this.setData({
        count:e.detail.value
    })
}
```

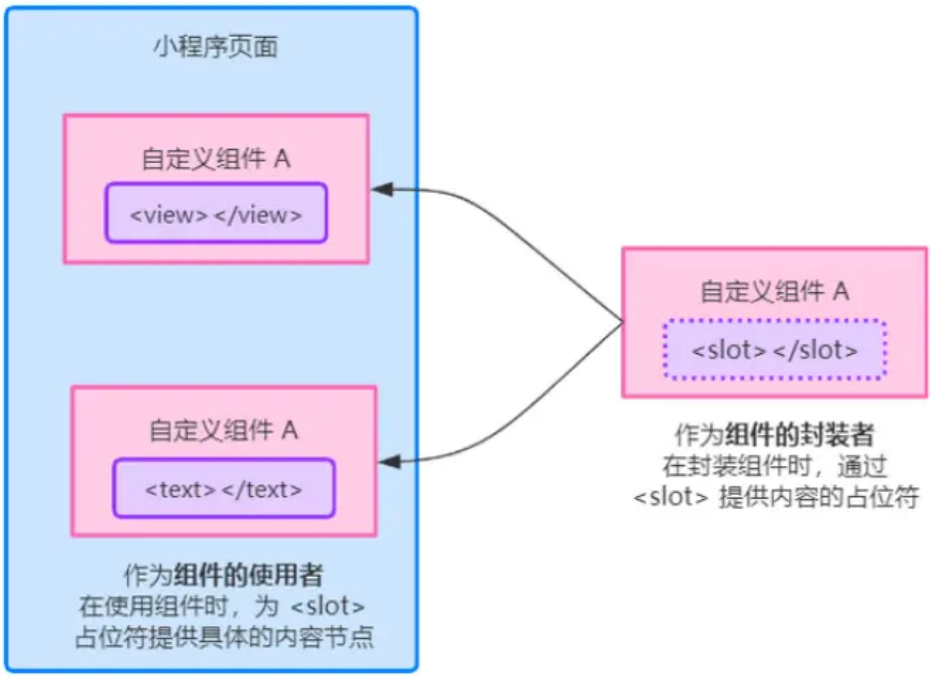
- 获得组件示例（通过 `this.selectComponent` 来获得子组件实例对象，这样直接访问子组件的任意方法和数据）

  可以在父组件里面调用 `this.selectComponent("id/类")`

```
getchild(){
    const child =this.selectComponent('.customA')
    child.setData({coutn:child.properties.count+1}) // 调用子组件的setData方法
    child.addCount();// 调用子组件的addCount方法。
}
```

插槽

什么是插槽？插槽是给组件预留的空间，当封装组件时往往会给组件使用插槽，插槽如何使用由父组件决定；在自定义组件的 wxml 结构中，可以提供一个 节点（插槽），**用于承载组件使用者提供的 wxml 结构**。



- 单个插槽

  小程序中，默认每个自定义组件中只允许使用一个插槽，这叫做：单个插槽。也就是说，对于不确定的内容，可以进行占位。

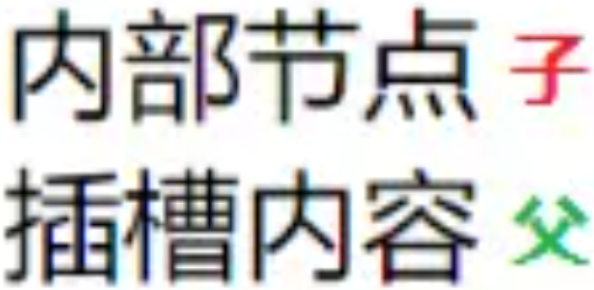
  子组件：

```
<view class="wrapper">
     <view>内部节点</view>
     // 使用插槽挖坑
     <slot></slot>
 </view>
```

  父组件：

```
 // 引用子组件
 <test>
   // 插入内容
   <view>插槽内容</view>
 </test>
```

  效果就是：



- 多个插槽

  如何使用多个插槽捏？在组件的options中进行配置

```
Component({
     options: {
         multipleSolts: true
         // 在组件定义时的选项中启用多slot支持
     }
 })
```

如何定位特定的插槽？ `子组件通过name属性` 配置插槽的名称， `父组件通过slot属性` 将节点进行填充。

子组件:

```
<view class="wapper">
   <slot name="before"></slot>
   <view>我是子组件</view>
   <slot name="after"></slot>
 </view>
```

父组件:

```
 // 引用子组件
 <test>
   <view slot="before">我在上</view>
   <view slot="after">我在下</view>
 </test>
```

6、全局数据共享——mobx

类似于vue中的vuex，可以用来实现状态的存储。解决组件之间数据共享的问题。

1.创建Mobx的Store实例

```
import { observable, action } from 'mobx-miniprogram'

// 数据仓库
export const store = observable({

  list: [], // 天气数据（包含列表和详情）

  // 设置天气列表，从网络上获取到数据之后调用
  setList: action(function (list) {
    this.list = list
  }),

})
```

2.将Store中的数据绑定到页面中

如果页面中需要使用Mobx的数据，需要调用 `createStoreBindings` 来将仓库中的数据绑定到页面数据中。

```
import { createStoreBindings } from 'mobx-miniprogram-bindings'
import { store } from './store'

Page({
  onLoad() {
    // 绑定 MobX store
    this.storeBindings = createStoreBindings(this, {
      store, // 需要绑定的数据仓库
      fields: ['list'], // 将 this.data.list 绑定为仓库中的 list ，即天气数据
      actions: ['setList'], // 将 this.setList 绑定为仓库中的 setList action
    })
    // 从服务器端读取数据
    wx.showLoading()
    wx.request({ // 请求网络数据
```

```
      // ...
      success: (data) => {
        wx.hideLoading()
        // 调用 setList action ，将数据写入 store
        this.setList(data)
      }
    })
  },
  onUnload() {
    // 解绑,如果不解绑会出现内存泄漏的问题。
    this.storeBindings.destroyStoreBindings()
  },
})
```

这样，可以在 wxml 中直接使用 list ：

```
<view class="item" wx:for="{{list}}" wx:key="date" data-index="{{index}}">
  <!— 这里可以使用 list 中的数据了！ —>
  <view class="title">{{item.date}} {{item.summary}}</view>
  <view class="abstract">{{item.temperature}}</view>
</view>
```

7、npm

npm是前端的包管理器，类似于maven。可以简单看成引入的第三方js。

小程序也可以使用npm，但是却和web端的不同，小程序中有如下限制：

- 不支持依赖于node.js内置库的包
- 不支持依赖于浏览器内置对象的包
- 不支持依赖于C++插件的包

如何安装：

微信小程序中使用vant组件库（超详细） - 掘金 (juejin.cn)